

AUTOMATED EPHEMERAL PREVIEW ENVIRONMENTS

USING AWS AND DEVOPS

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT



Under the guidance of

Mrs. Sumana Bethala, APSSDC

Mr. Juttuka LovaBabu, APSSDC

Submitted by

AVVARU LAKSHMI VENKATA SHASHANK, B211432, RGUKT BASAR (Batch-2)

BATTU SIVA KOUSHIK, 23KT1A05D8, PSCMR CET (Batch-1)

SHAIK MEHTAJ, O210858, RGUKT ONGOLE (Batch-2)

BANKA PRANITHA RANI, 23KT1A05D7, PSCMR CET (Batch-1)

MADINENI VENKATA ROHITH, O211076, RGUKT ONGOLE (Batch-2)

Acknowledgement

I would like to express my heartfelt gratitude to all those who have contributed to the successful completion of my summer internship project at **Andhra Pradesh Skill Development Corporation (APSSDC)**. This opportunity has been an enriching and transformative experience for me, and I am truly thankful for the support, guidance, and encouragement I have received along the way.

First and foremost, I extend my sincere regards to Mrs Sumana Bethala and Mr Anil Kumar Varanasi, my supervisors and mentors, for providing me with valuable insights, constant guidance, and unwavering support throughout the duration of the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

I would like to thank the entire team at **Andhra Pradesh Skill Development Corporation (APSSDC)** for fostering a collaborative and innovative environment. The camaraderie, knowledge sharing, and feedback I received from my colleagues significantly contributed to the development and success of this project.

In conclusion, I am honoured to have been a part of this internship program, and I look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Thank you.

Sincerely,

Battu Siva Koushik, 23KT1A05D8, sivakoushikbattu01@gmail.com.

Banka Pranitha Rani, 23KT1A05D7, pranitharani518@gmail.com.

Avvaru Lakshmi Venkata Shashank, B211432, alvshashank@gmail.com.

Shaik Mehtaj, O210858, o210858@rguktong.ac.in.

Madineni Venkata Rohith, O211076, rohithmadineni29@gmail.com.

Abstract

In modern software development lifecycles, shared staging environments frequently create significant deployment bottlenecks. When multiple developers test concurrent features, code overlap and resource saturation often lead to blocked pipelines and testing inaccuracies. **Automated Ephemeral Preview Environments** is a comprehensive DevOps infrastructure project engineered to eliminate staging friction by dynamically provisioning isolated, temporary hosting environments tied directly to individual GitHub Pull Requests.

The architecture enables concurrent development teams to operate within a unified continuous deployment framework. The system intercepts Pull Request events to automatically containerize the application, push encrypted image layers to a secure registry, and deploy the isolated container to a shared host using deterministic port mapping. Dynamic reverse-proxy injection enables instant access via uniquely generated URLs, while minimizing transactional pipeline latency.

The platform relies on a fault-tolerant, industry-standard technology stack. The automation engine is driven by **GitHub Actions**, which orchestrates **Docker** builds and manages **Amazon ECR** authentication. Deployment is routed to an **AWS EC2** host where a master **Nginx** container processes wildcard DNS resolution to route incoming traffic dynamically without dropping active connections.

This project successfully addresses primary infrastructure scaling pain points—eliminating port collisions, minimizing compute redundancy, and ensuring zero-cost resource management. By implementing an event-driven teardown sequence that permanently wipes inactive containers and image layers upon Pull Request closure, this architecture demonstrates a highly scalable, economically optimized approach to modern CI/CD GitOps engineering.

Contents

Acknowledgement	ii
Abstract	iii
List of Figures	vi
List of Abbreviations and Symbols Used	vii
1 Introduction	1
1.1 Introduction	1
1.2 Motivation	1
1.3 Problem Definition	2
1.4 Objective of the Project	2
2 Literature Review	4
2.1 Evolution of Continuous Deployment	4
2.2 Containerization and Process Isolation	4
2.3 Dynamic Routing and Fault Tolerance	5
2.4 Comparative Analysis of Deployments	5
2.5 Summary	5
3 Analysis	6
3.1 System Requirement Analysis	6
3.1.1 Functional Requirements	6
3.1.2 Non-Functional Requirements	7
3.1.3 Hardware and Software Requirements	8

4	System Design	9
4.1	Visualizing the System Architecture	9
4.1.1	Deployment Pipeline Flow (Architecture Diagram)	9
4.1.2	The Execution Sequence (Sequence Diagram)	10
4.2	Organizing the Network (Routing Design)	11
4.2.1	The Network Routing Schema	11
4.3	How Information Moves (Execution Flow)	11
4.4	Data Storage & Artifact Architecture	12
5	Implementation	13
5.1	Project Structure	13
5.2	GitHub Actions Orchestration	14
5.2.1	Deployment Pipeline Workflow	14
5.3	Core Module Implementation	15
5.3.1	Remote EC2 Execution and Port Allocation	15
5.3.2	Dynamic Nginx Routing and URL Generation	15
5.3.3	Automated Teardown and Resource Cleanup	16
6	Testing	18
6.1	System and Integration Testing	18
6.2	Security Paradigms	19
7	Conclusion	20
	References	20

List of Figures

4.1	Architecture Diagram: CI/CD interactions between GitHub, AWS ECR, and EC2	10
4.2	Sequence Diagram: The timeline from Git push to live URL availability	10
4.3	Network Schema: Route resolution from public internet to isolated Docker containers	11
5.1	GitHub Actions execution log displaying a successful Docker build and deployment	14
5.2	Automated GitHub Bot commenting the generated dynamic URL on the Pull Request	16
5.3	The live ephemeral application successfully resolving via the dynamic Wildcard DNS	16
5.4	Teardown pipeline successfully destroying containers and ECR images upon PR closure	17

List of Abbreviations and Symbols Used

AWS	Amazon Web Services
CI/CD	Continuous Integration / Continuous Delivery
EC2	Elastic Compute Cloud
ECR	Elastic Container Registry
DNS	Domain Name System
IAM	Identity and Access Management
SSH	Secure Shell
VPC	Virtual Private Cloud
API	Application Programming Interface
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IP	Internet Protocol
TCP	Transmission Control Protocol
IaC	Infrastructure as Code

Chapter 1

Introduction

1.1 INTRODUCTION

In agile software engineering environments, the speed at which developers can verify and merge code directly dictates project velocity. Traditional infrastructure models rely on a single, static staging server where all feature branches are merged for testing. This architecture inherently creates a bottleneck: if two developers push conflicting updates, or if the staging database crashes, the entire testing pipeline is halted. To resolve this, this project implements **Automated Ephemeral Preview Environments**, a GitOps-based paradigm that spins up an isolated, fully functional copy of the application for every single feature branch.

This system is engineered to serve software engineering teams. It enables developers to work independently, opening Pull Requests that immediately trigger a GitHub Actions pipeline. The pipeline containerizes the new code and deploys it to a dynamic URL, allowing quality assurance teams to test the specific feature in total isolation before it ever touches the primary staging or production servers.

1.2 MOTIVATION

Infrastructure overhead and cloud costs are notoriously difficult to optimize. Allocating dedicated servers for every developer is financially unviable for most mid-tier organizations. However, forcing developers to queue for access to a single staging environment reduces productivity and increases time-to-market.

The need to solve deployment concurrency transparently while maintaining strict cost con-

trols strongly motivated this architecture. By utilizing Docker for lightweight containerization, GitHub Actions for serverless pipeline execution, and event-driven cleanup scripts, this infrastructure provides enterprise-grade isolated testing environments while ensuring idle resources are destroyed to maintain a near-zero cost footprint.

1.3 PROBLEM DEFINITION

Currently, engineering teams face significant technical hurdles in deployment:

- **Testing Bottlenecks:** Shared staging servers block concurrent feature testing.
- **Port Collisions:** Attempting to run multiple application versions on a single host manually results in TCP port conflicts and service failures.
- **Resource Orphanage:** Temporary testing servers are frequently left running by mistake, accruing massive cloud billing costs.
- **Context Switching:** Developers waste time configuring local database states to review peer code instead of simply clicking a live URL.

This architecture was explicitly designed to address and eliminate these exact infrastructural bottlenecks.

1.4 OBJECTIVE OF THE PROJECT

The underlying objective of this project is to provide a robust, automated DevOps pipeline that brings fault-tolerant continuous deployment capabilities to development teams. The key objectives include:

1. Implementing an automated **GitHub Actions workflow** to intercept Pull Request triggers and execute CI/CD logic.
2. Utilizing **Docker** to containerize applications, pushing secure layers to **Amazon ECR**.

3. Architecting a dynamic routing system via a master **Nginx** proxy to facilitate wildcard DNS mapping to isolated container ports.
4. Calculating deterministic port mappings based on GitHub Pull Request IDs to mathematically eliminate network collisions on the **AWS EC2** host.
5. Developing an event-driven **Teardown Script** that automatically destroys containers, deletes proxy routes, and prunes unused images to guarantee zero idle resource consumption.

Chapter 2

Literature Review

The evolution of deployment architectures has transitioned from manual FTP uploads to highly concurrent, automated distributed pipelines. This chapter explores the existing research and technological methodologies that inform the architecture of this project, focusing on GitOps, Container Orchestration, and Reverse Proxy dynamics.

2.1 EVOLUTION OF CONTINUOUS DEPLOYMENT

Early deployment systems relied on monolithic, long-lived servers managed manually by system administrators. Research into infrastructure-as-code (IaC) indicates that manual configuration inevitably leads to "configuration drift," where staging environments slowly fall out of sync with production. Modern CI/CD pipelines resolve this by codifying the deployment steps in version control, ensuring every deployment is strictly reproducible.

2.2 CONTAINERIZATION AND PROCESS ISOLATION

To solve dependency conflicts between different application versions, modern systems have shifted toward containerization.

- **Docker Engine:** Research confirms that leveraging Linux kernel namespaces (cgroups) allows Docker to run multiple isolated instances of an application on a shared kernel with minimal overhead. This is vastly superior to provisioning distinct Virtual Machines (VMs) for every preview environment, which incurs heavy CPU and memory penalties.
- **Image Layering:** By optimizing the `Dockerfile` to cache dependencies before copying source code, the pipeline drastically reduces build times, improving the feedback loop

latency for developers.

2.3 DYNAMIC ROUTING AND FAULT TOLERANCE

Deploying multiple containers to a single host requires advanced traffic management.

- **Reverse Proxies:** Nginx operates as a high-performance asynchronous edge proxy. Instead of exposing raw Docker ports to the public internet, Nginx intercepts all port 80 traffic and evaluates the HTTP `Host` header.
- **Zero-Downtime Reloads:** A critical component of fault-tolerant routing is the ability to update configuration files without dropping active user connections. Nginx allows for seamless worker process reloading (`nginx -s reload`), making it ideal for an environment where routing tables change dynamically every time a Pull Request is opened or closed.

2.4 COMPARATIVE ANALYSIS OF DEPLOYMENTS

Table 2.1: Comparison of Staging Architectures

Feature	Traditional Staging	Ephemeral GitOps
Environment	Single Shared Server	Isolated per Pull Request
Concurrency	Blocked (Queue Required)	Infinite Horizontal Scaling
Routing	Static IP / Subdomain	Dynamic Wildcard DNS
Cost Model	24/7 Uptime Billing	Zero-Cost via Auto-Teardown
Configuration	Manual Server Admin	Automated YAML Pipelines

2.5 SUMMARY

The literature confirms that the integration of **GitHub Actions** for orchestration, **Docker** for isolation, and **Nginx** for dynamic routing represents the current industry standard for scalable preview environments. This project synthesizes these findings to create a resilient, fault-tolerant infrastructure pipeline that prioritizes both developer velocity and absolute cost efficiency.

Chapter 3

Analysis

3.1 SYSTEM REQUIREMENT ANALYSIS

This GitOps architecture is conceptualized to support dynamic engineering teams. To clearly define the scope and architecture, assessing the system through detailed **functional** and **non-functional** requirements is essential.

3.1.1 Functional Requirements

1. Deployment Trigger Requirements

- The pipeline must execute automatically when a developer opens or synchronizes a GitHub Pull Request.
- The system must securely authenticate with AWS IAM without exposing static credentials in the source code.
- The workflow must compile a Docker image and tag it specifically using the PR numerical identifier.

2. Host Execution Requirements

- The EC2 host must pull the latest container image from the private ECR registry.
- The deployment script must calculate a unique network port (e.g., $8000 + PR_ID$) to bind the container safely.

- A dynamic Nginx configuration file must be generated on the host and injected into the proxy network.
- A live, clickable URL must be posted back to the GitHub PR timeline via an automated bot comment.

3. Teardown Requirements

- The cleanup workflow must exclusively trigger upon PR closure or merge events.
- The system must execute targeted removal of the specific container and Nginx configuration file without disrupting other active PR environments on the same host.
- Remote AWS ECR image layers must be purged via the AWS CLI to prevent storage cost accumulation.

3.1.2 Non-Functional Requirements

- **Isolation (Concurrency):** The environments must operate in strict silos. A fatal crash in PR-1 must have zero impact on the execution of PR-2 on the same host machine.
- **Fault-Tolerance:** The Nginx routing table reloads must execute gracefully, ensuring that external traffic to existing environments is not dropped during new deployments.
- **Cost Optimization:** The architecture strictly demands automated destruction of idle resources, guaranteeing a near-zero AWS footprint outside of active development hours.
- **Pipeline Latency:** The Dockerfile and caching strategies must be optimized to ensure the complete cycle—from code push to live URL—executes in under three minutes.

3.1.3 Hardware and Software Requirements

Software Requirements

- **Version Control & CI/CD:** GitHub, GitHub Actions.
- **Containerization:** Docker Engine, Docker Compose.
- **Cloud Infrastructure:** AWS EC2 (Ubuntu Linux environment), AWS ECR, AWS IAM.
- **Reverse Proxy & Routing:** Nginx.
- **Scripting & Execution:** Bash, AWS CLI, SSH.
- **Domain & DNS:** DuckDNS (Wildcard Domain Resolution).
- **Environment:** Linux/Unix ecosystem.

Chapter 4

System Design

System design serves as the technical blueprint for the pipeline. It defines the overall structure, how distinct cloud components communicate, and how network traffic routes from a public DNS record down to an isolated Docker container. The system utilizes a decoupled architecture: a remote execution pipeline for continuous integration and a secure host network for isolated testing.

4.1 VISUALIZING THE SYSTEM ARCHITECTURE

To make the infrastructure logic transparent and manageable, we utilize architectural diagrams. These act as blueprints, illustrating the functional boundaries and logical connections between system components.

4.1.1 Deployment Pipeline Flow (Architecture Diagram)

The architecture diagram defines the boundaries for the systems interacting within the deployment pipeline:

- **Version Control (GitHub):** Detects repository state changes and initiates serverless compute runners.
- **Artifact Storage (AWS ECR):** Acts as the secure, private vault for compiled Docker image layers.
- **Compute Engine (AWS EC2):** The physical host that executes the runtimes, manages internal bridge networks, and controls the Nginx proxy layer.

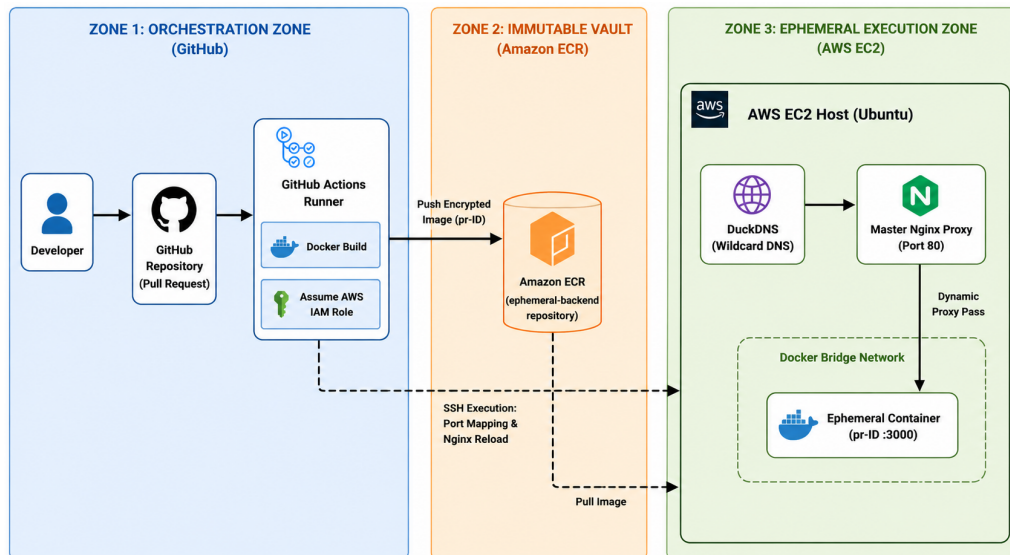


Figure 4.1: Architecture Diagram: CI/CD interactions between GitHub, AWS ECR, and EC2

4.1.2 The Execution Sequence (Sequence Diagram)

This diagram illustrates the time-ordered timeline of a GitOps deployment. It specifically highlights the **Event-Driven** mechanism: the moment a developer pushes code, the pipeline dictates a precise sequence of authentication, compilation, remote execution, and routing table manipulation.

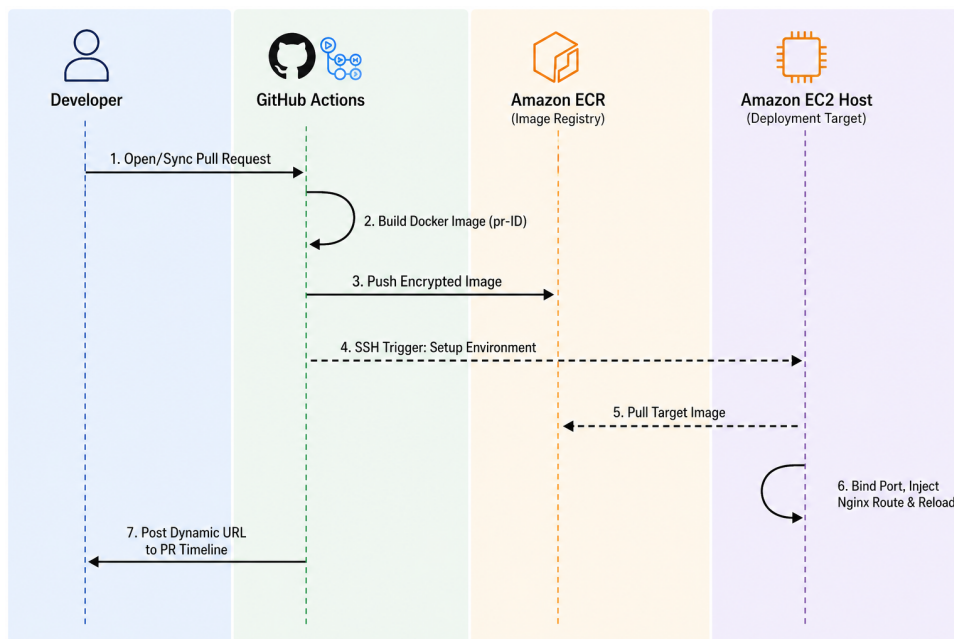


Figure 4.2: Sequence Diagram: The timeline from Git push to live URL availability

4.2 ORGANIZING THE NETWORK (ROUTING DESIGN)

To ensure fault-tolerant routing and prevent port collisions during high-traffic testing periods, the system requires a highly organized network topology.

4.2.1 The Network Routing Schema

The network schema acts as a map for the system's traffic layer. It details how an external HTTP request resolves through a Wildcard DNS provider (DuckDNS), enters the AWS Virtual Private Cloud (VPC), hits the primary Nginx container, and is mathematically forwarded to the correct Docker bridge network port based on the HTTP Host header.

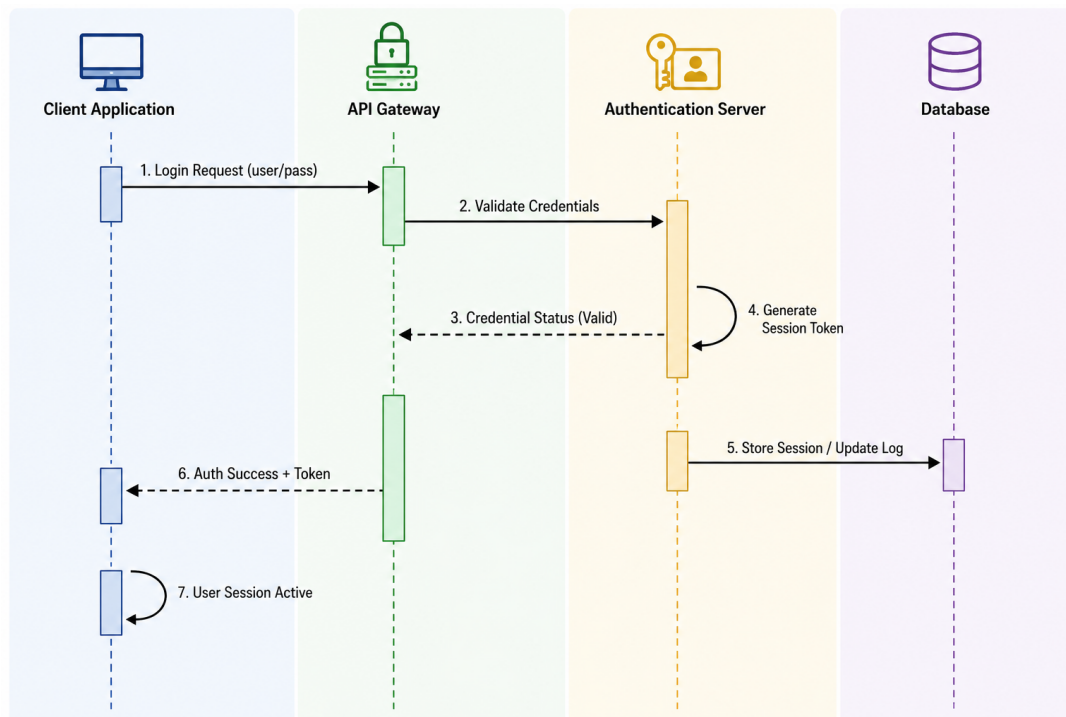


Figure 4.3: Network Schema: Route resolution from public internet to isolated Docker containers

4.3 HOW INFORMATION MOVES (EXECUTION FLOW)

The movement of deployment instructions and network traffic follows a strict logical loop to maintain synchronization:

1. **Input / Trigger:** A developer opens or synchronizes a GitHub Pull Request.

2. **Processing:**

- GitHub Actions authenticates with AWS IAM and compiles the Docker image.
- The image is pushed to the Amazon ECR registry.
- An SSH tunnel is established to the EC2 host to calculate deterministic port mappings and execute the container.
- The Nginx reverse proxy configuration is dynamically generated and gracefully reloaded.

3. **Output:** The staging environment goes live globally, and the dynamic URL is posted back to the GitHub PR timeline for the developer.

4.4 DATA STORAGE & ARTIFACT ARCHITECTURE

To optimize for speed, security, and immutability, data is categorized and stored in specialized environments:

- **Source of Truth (GitHub):** The permanent repository for application source code, infrastructure YAML workflows, and encrypted CI/CD secrets.
- **Artifact Registry (Amazon ECR):** High-speed storage for compiled Docker images, allowing rapid pulls directly to the EC2 host within the AWS backbone.
- **Ephemeral Execution (EC2 Host):** Manages the volatile state. Containers and Nginx routing files exist here only temporarily until the corresponding Pull Request is closed and the teardown script purges them.

Chapter 5

Implementation

The implementation of the **Automated Ephemeral Preview Environments** follows a strict Infrastructure-as-Code (IaC) and GitOps methodology. By decoupling the deployment orchestration from the application source code, the system ensures that complex concurrency, routing, and lifecycle management are handled seamlessly by cloud runners and automated host scripts.

5.1 PROJECT STRUCTURE

The repository is structured to encapsulate both the application logic (a MERN/Node.js backend) and the infrastructure definitions required to deploy it dynamically.

```
ephemeral-env-project/
├── .github/
│   ├── workflows/
│   ├── deploy.yml (CI/CD Pipeline triggered on PR open/sync)
│   └── teardown.yml (Cleanup script triggered on PR closed)
├── app/ (Application Source Code)
│   ├── src/ (Controllers, Routes, Models)
│   ├── Dockerfile (Multi-stage optimized build instructions)
│   ├── package.json (Node.js dependencies)
│   └── .dockerignore (Build context optimizations)
├── nginx/ (Master Proxy Configuration)
│   └── docker-compose.yml (Initialization of the Nginx master container)
├── scripts/
└── host-setup.sh (Bash logic executed on the AWS EC2 instance)
```

5.2 GITHUB ACTIONS ORCHESTRATION

To maintain deployment consistency and eliminate manual intervention, the entire deployment lifecycle is governed by **GitHub Actions**.

5.2.1 Deployment Pipeline Workflow

The primary CI/CD suite (‘deploy.yml’) is triggered automatically on ‘pull_request’ events (specifically ‘opened’ and ‘synchronize’). The pipeline executes the following sequential stages:

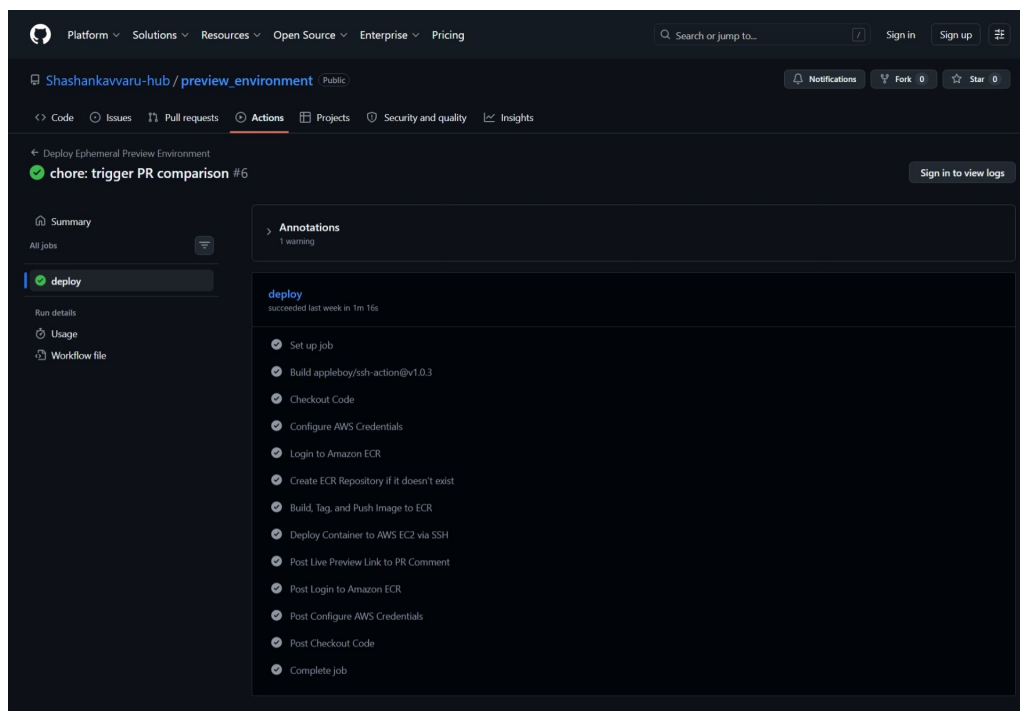


Figure 5.1: GitHub Actions execution log displaying a successful Docker build and deployment

- **Environment Initialization:** Injects the unique GitHub Pull Request ID (‘PR_ID’) into the environment variables.
- **AWS IAM Authentication:** Utilizes encrypted GitHub Secrets to securely assume a predefined IAM role, granting temporary ‘ECRPowerUser’ access without hardcoding static AWS credentials.

- **Docker Compilation:** Executes the ‘docker build‘ command, tagging the resulting artifact dynamically (e.g., ‘ephemeral-backend:pr-1‘).
- **Registry Push:** Pushes the compiled, encrypted image layer to the private Amazon ECR repository.

5.3 CORE MODULE IMPLEMENTATION

5.3.1 Remote EC2 Execution and Port Allocation

Once the image resides in Amazon ECR, the GitHub Actions runner establishes a secure SSH tunnel to the AWS EC2 host. The critical task here is avoiding network collisions.

- **Deterministic Port Math:** The bash script receives the ‘PR_ID‘ and calculates a distinct host port mapping using the formula ‘ $((8000 + PR_ID))$ ’. If PR #5 is opened, the host dynamically binds to port 8005.
- **Container Provisioning:** The script executes ‘docker run -d‘, pulling the specific PR image from ECR and connecting it securely to the isolated internal Docker bridge network.

5.3.2 Dynamic Nginx Routing and URL Generation

Purpose: To route traffic from a public wildcard domain to the isolated container without exposing internal ports.

- **Configuration Injection:** The bash script generates a unique ‘.conf‘ file and injects it into the master Nginx container’s volume. This file explicitly maps the ‘server_name‘ (e.g., ‘pr-5.shashank-devops.duckdns.org‘) to the calculated internal ‘proxy_pass‘.
- **Graceful Reload:** A ‘docker exec master-nginx nginx -s reload‘ command is issued, instantly updating the routing tables without dropping any active user connections across other PR environments.

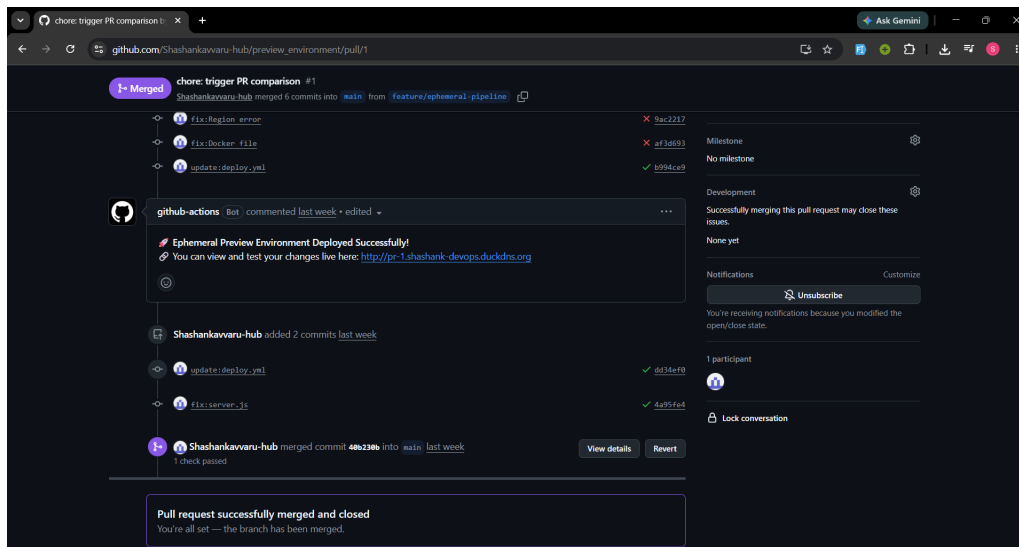


Figure 5.2: Automated GitHub Bot commenting the generated dynamic URL on the Pull Request

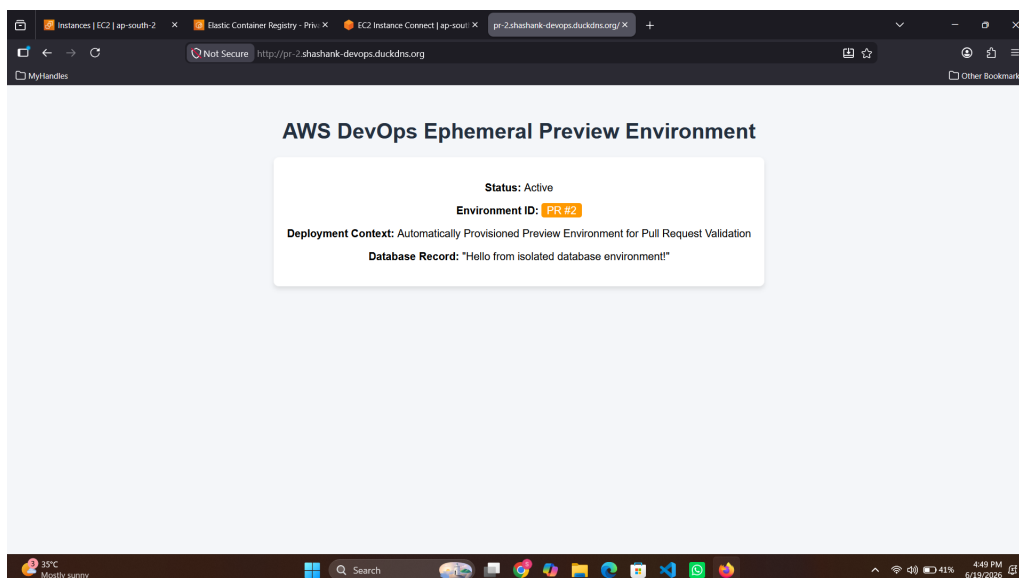


Figure 5.3: The live ephemeral application successfully resolving via the dynamic Wildcard DNS

5.3.3 Automated Teardown and Resource Cleanup

To guarantee a zero-cost infrastructure footprint, the system implements a strict garbage collection mechanism via ‘teardown.yml’.

- When a developer clicks ”Merge” or closes a Pull Request, the teardown pipeline intercepts the event.

- It SSHs into the EC2 host and executes a ‘docker stop’ and ‘docker rm’ targeted specifically at the closed PR’s container.
- The script deletes the associated Nginx ‘.conf’ file and reloads the proxy to clear the dead route.
- Finally, an AWS CLI command (‘aws ecr batch-delete-image’) is executed to permanently destroy the heavy image layer in the remote repository, preventing hidden storage fees.

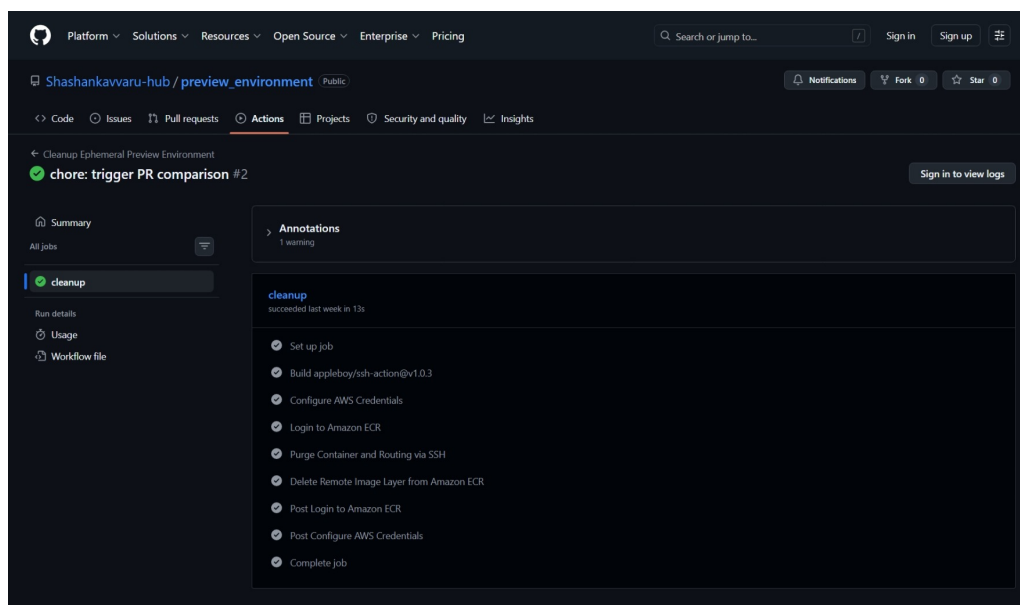


Figure 5.4: Teardown pipeline successfully destroying containers and ECR images upon PR closure

Chapter 6

Testing

Evaluating robustness inside an automated infrastructure environment necessitates rigid verification strategies covering pipeline execution, network isolation, and resource cleanup integrity. Ensure the system reliably builds, routes, and destroys environments without human intervention.

6.1 SYSTEM AND INTEGRATION TESTING

System parameters were validated heavily against logical overlap parameters alongside standard workflow execution tracking.

1. **Concurrency and Port Collision Tests:** Deployed multiple feature branch Pull Requests simultaneously to explicitly validate the bash arithmetic logic. The testing confirmed that rapid sequential deployments successfully bound to unique host ports (e.g., 8001, 8002) without Docker daemon rejection or TCP allocation errors.
2. **Dynamic Routing Resolution:** Validated wildcard routing via DuckDNS, ensuring external HTTP requests successfully traversed the public internet, penetrated the AWS Security Group, and were accurately forwarded by the master Nginx proxy to the isolated internal Docker containers.
3. **Teardown Lifecycle Verification:** Merged and closed PRs while actively monitoring the AWS EC2 storage and ECR registry dashboard. Confirmed that webhook callbacks successfully triggered the teardown script, terminating the specific Docker container, deleting the '.conf' route natively without disruption to other active environments, and purging the remote ECR image layer.

6.2 SECURITY PARADIGMS

- **IAM Principle of Least Privilege:** The IAM user configured for the GitHub Actions runner is strictly limited to the ‘AmazonEC2ContainerRegistryPowerUser’ policy. It possesses zero permissions to modify or destroy EC2 instances, ensuring host safety if CI/CD tokens are ever compromised.
- **Network Isolation:** Ephemeral Docker containers are deployed explicitly to an internal Docker Bridge Network and do not publish ports directly to the host’s public IP (‘0.0.0.0’). All external traffic must funnel strictly through the Nginx proxy via Port 80, neutralizing direct port-scanning attacks.
- **Encrypted Secrets Management:** Long-term AWS credentials and the EC2 SSH PEM key are injected dynamically into the runtime environment via encrypted GitHub Actions Secrets. They are never materialized or committed to the version control repository.

Chapter 7

Conclusion

The **Automated Ephemeral Preview Environments** architecture actively mitigates the primary deployment bottlenecks inherent in modern agile software development. By strictly leveraging immutable Docker containerization interlaid harmoniously with dynamic Nginx reverse-proxy generation, this infrastructure bridges critical testing gaps distinctly, transforming manual staging processes into a highly resilient, automated continuous delivery pipeline.

By engineering explicitly for event-driven GitOps workflows, the resulting system yields highly impressive developer velocity. It allows isolated, concurrent testing of feature branches, effectively eliminating the traditional deployment queues and code-overlap issues associated with static, shared staging environments. The implementation of deterministic port allocation logic ensures network collisions are structurally impossible, even under high-frequency Pull Request deployment loads.

Furthermore, the resolution of cloud resource bloat through deeply integrated, event-triggered teardown scripts decisively cements this architecture as an enterprise-grade conceptual framework. The pipeline guarantees a zero-cost idle footprint by purging containers and remote image layers upon Pull Request closure, proving that high-availability CI/CD deployment strategies can be executed without sacrificing strict financial optimization.

References

- [1] GitHub Docs, *Understanding GitHub Actions*.
Available at: <https://docs.github.com/en/actions>
- [2] Docker Documentation, *Docker Overview and Reference*.
Available at: <https://docs.docker.com/>
- [3] Amazon Web Services Documentation, *Amazon Elastic Compute Cloud (EC2)*.
Available at: <https://docs.aws.amazon.com/ec2/>
- [4] Amazon Web Services Documentation, *Amazon Elastic Container Registry (ECR)*.
Available at: <https://docs.aws.amazon.com/AmazonECR/latest/userguide/>
- [5] NGINX Documentation, *NGINX Reverse Proxy and Configuration*.
Available at: <https://nginx.org/en/docs/>
- [6] GNU Project, *Bash Reference Manual*.
Available at: <https://www.gnu.org/software/bash/manual/>
- [7] DuckDNS, *Free Dynamic DNS Service*.
Available at: <https://www.duckdns.org/>
- [8] GitLab Documentation, *What is GitOps?*.
Available at: <https://about.gitlab.com/topics/gitops/>
- [9] Appleboy, *GitHub Action for Executing Remote SSH Commands*.
Available at: <https://github.com/appleboy/ssh-action>